

Programa · Comisión 2026

AI Engineering

Clase 3:

Persistencia de datos y vector DBs

Objetivos

1. Explicar la diferencia entre representaciones discretas (One-Hot) y densas (Embeddings).
2. Elegir la métrica de similitud adecuada (Coseno, Euclídea o Producto Punto) según el caso.
3. Diseñar una estrategia de chunking y overlap para no perder contexto semántico.
4. Implementar operaciones CRUD sobre una base de datos vectorial local con ChromaDB.
5. Construir un pipeline RAG completo: ingesta, recuperación y generación fundamentada.

Agenda

1. Embeddings y similitud: la geometría del lenguaje.
2. Estrategias de chunking y preprocesamiento de documentos.
3. Persistencia local con ChromaDB: operaciones CRUD.
4. Pre-entrega 3: sistema de recuperación semántica local (RAG).

**¡Esta clase va a ser
grabada!**



01

Embeddings y similitud

¿Qué es un

Embedding

El lenguaje convertido en geometría

Un embedding es una lista de números (un vector) que captura el significado de un fragmento de texto.

vs. One-Hot Encoding: cada palabra era una posición en un vector gigante de ceros, sin noción de significado.

Con embeddings, la distancia entre vectores refleja qué tan cerca están dos ideas.

La Geometría de la Similitud

3 métricas para medir cercanía

- **Similitud Coseno:** mide el ángulo entre vectores; ignora la longitud del texto. Es el estándar en RAG.
- **Distancia Euclídea (L2):** mide la distancia en línea recta; sensible a la magnitud del vector.
- **Producto Punto:** combina magnitud y ángulo; útil cuando la intensidad del vector importa.

Regla de oro: con modelos de OpenAI o HuggingFace, la Similitud Coseno es casi siempre la elección correcta.

Estáticos vs. Contextuales

La polisemia resuelta por contexto

Estáticos (Word2Vec, GloVe): cada palabra tiene un único vector. "Banco" (sentarse) y "banco" (financiero) son idénticos.

Contextuales (BERT, Ada, Voyage): la misma palabra genera vectores distintos según el contexto que la rodea.

El modelo contextual entiende que en "me senté en el banco", la palabra refiere a mobiliario urbano.

De el texto al vector

Un embedding en acción

Input: el modelo `text-embedding-3-small` de OpenAI toma un texto como string.

Proceso: el modelo lo transforma en un vector denso de 1536 dimensiones.

Comparación: se calcula la similitud coseno entre vectores para medir relación semántica.

Resultado: $\text{sim}(\text{"disco lleno"}, \text{"sin espacio"}) \approx 0.85$ vs. $\text{sim}(\text{"disco lleno"}, \text{"me gustan las manzanas"}) \approx 0.15$.

Errores Comunes con Embeddings

Comprensión total

Los embeddings capturan correlaciones estadísticas, no lógica pura: pueden confundir la negación.

Ignorar el límite de tokens

Cada modelo tiene una ventana de contexto; un PDF de 100 páginas se truncará o perderá precisión.

Mezclar modelos

Nunca compares un embedding de OpenAI con uno de HuggingFace: viven en espacios vectoriales distintos.

02

Chunking y preprocesamiento

¿Por qué fragmentar?

La ventana de contexto tiene un límite

Los modelos de embedding tienen una ventana de contexto limitada; un documento entero pierde matices.

El objetivo del chunking: crear unidades de información atómicas y semánticamente coherentes.

Antes de dividir hay que limpiar: encabezados repetidos, saltos de línea y caracteres basura ensucian el embedding.

Estrategias de

Fixed-size

Divide cada X caracteres o tokens. Simple, pero puede cortar una oración o concepto por la mitad.

Recursive Splitting

El estándar de LangChain: prueba párrafos, luego saltos de línea, luego espacios, luego caracteres.

¿Cuál usar?

Recursive Character Splitting mantiene unidos párrafos y oraciones en la mayoría de los casos reales.

El proceso de limpieza

Preprocessing antes de fragmentar

Los documentos reales (PDFs, HTMLs) traen ruido: encabezados, pies de página, saltos de línea dobles.

También rutas de archivo, metadatos basura y caracteres especiales de una mala extracción de tablas.

Mal: un chunk con un pie de página en medio de una frase. Bien: texto limpio antes de fragmentar y vectorizar.

La jerarquía de separadores

Cómo decide dónde cortar

Recursive Character Splitting prueba una jerarquía de separadores, en este orden.

- **Nivel 1: Párrafos (\n\n).**
- **Nivel 2: Saltos de línea (\n).**
- **Nivel 3: Espacios ().**

Nivel 4 (último recurso): caracteres individuales.

Overlap: el solapamiento

Que nada se pierda en el corte

Overlap: al dividir, el final del Chunk 1 se repite al inicio del Chunk 2.

Motivación: si un concepto clave queda justo en el límite entre dos fragmentos, el overlap preserva el contexto.

- El contexto se mantiene en ambos fragmentos, sin importar por dónde se "cortó" el texto.

Práctica común: chunks de 500 tokens con 50 de overlap.

Momentos de la Clase:

Para recordar

Coder Tips

Material ampliado

Cuestionario de tarea

#FindTheBug

Chunks muy pequeños pierden contexto global; medir en caracteres en vez de tokens es el error #1. Validá siempre con tiktoken.

Práctica - Unidad 2

El DocumentProcessor

Objetivo: construir un DocumentProcessor que limpie y fragmente texto por tokens, no por caracteres.

Manos a la obra:

- Implementará clean_text con regex para eliminar ruido y espacios duplicados.
- Configurará RecursiveCharacterTextSplitter con chunk_size=500 y chunk_overlap=50, midiendo con tiktoken.

Resultado: un pipeline de limpieza → fragmentación listo para poblar tu base vectorial.

Break,

andá por un cafecito
o mate, ¡y volvemos!



03

Persistencia con ChromaDB

¿Por qué ChromaDB local?

Memoria que sobrevive al reinicio

Los embeddings son volátiles si solo viven en RAM: la memoria del sistema no debe perderse al reiniciar.

ChromaDB es una base de datos vectorial de código abierto, simple, sin necesidad de Docker por defecto.

Guarda los datos en disco y permite operaciones CRUD sobre el conocimiento de tu agente.

CRUD en el espacio

Create (Upsert)

Si el texto cambia, el vector se recalcula. Upsert crea o actualiza según si el ID ya existe.

Read: get() vs query()

get() busca por IDs o metadatos exactos. query() busca por similitud semántica: el corazón de RAG.

Update y Delete

Update dispara una nueva vectorización; Delete elimina vectores que ya no son relevantes.

Metadatos y filtrado

Etiquetas para buscar mejor

Los metadatos son etiquetas adjuntas a los vectores, por ejemplo autor, fecha o fuente del documento.

Sirven para filtrar resultados antes de la búsqueda vectorial, acotando el espacio de búsqueda.

También permiten rastrear el origen de la información que usa el LLM, para citar sus fuentes.

Regla clave: nunca consultes una colección con un modelo de embeddings distinto al que la creó.



Práctica - Unidad 3

El VectorMemoryManager

Objetivo: encapsular las operaciones CRUD de ChromaDB en una clase robusta.

Manos a la obra:

- Inicializá un PersistentClient con una ruta de disco y una colección.
- Implementá `upsert_documents` y `semantic_search` usando `collection.query()`.

Resultado: tu `VectorMemoryManager` listo para poblar y consultar tu base vectorial.

Pre Entrega

Pre-Entrega 3

¿Qué debes construir?

Pre-Entrega 3

Debes entregar un script o notebook que implemente un flujo End-to-End de RAG.

- **Ingesta:** fragmenta documentos `.txt` o `.md` y los persiste en una colección de ChromaDB.
- **Retriever:** convierte la pregunta del usuario en embedding y recupera los fragmentos más relevantes.

Generación Grounded: una cadena LCEL que responde solo con el contexto recuperado, o dice "no lo sé".

Tu misión

El entregable



Duración estimada: ~2,5 horas



Entrega: repositorio de GitHub con ingesta, cadena RAG y README

Construí un sistema RAG local que responda preguntas basándose exclusivamente en tus documentos, sin alucinar.

Pasos

1. Instalá langchain, chromadb, tu proveedor de LLM y pydantic en un entorno virtual.
2. Cargá archivos de /data, aplicá chunking (mínimo 500 tokens, 50 de overlap) y guardalos en ChromaDB.
3. Definí `get_rag_response(query)`: busca por similitud, arma el prompt y llama al LLM de forma asíncrona.
4. Parseá la respuesta a un modelo Pydantic con texto y referencias.
5. Probá una pregunta con respuesta en los documentos y una "pregunta trampa" sin respuesta.

Errores comunes a evitar

Antes de entregar, revisá esto

El "Contexto Infinito": no pases 50 fragmentos al LLM; mantené un top_k de entre 3 y 5.

- Embeddings no coincidentes: indexar con un modelo y consultar con otro genera resultados aleatorios.
- Falta de persistencia: verificá si la base ya existe antes de reindexar todo desde cero.
- Seguridad: nunca subas tus llaves de API; usá variables de entorno .env.

¿Cómo se evalúa? 100 pts

Chunking e Ingesta · 25%

Procesar documentos aplicando fragmentación que preserve el contexto semántico.

Arquitectura Asíncrona · 35%

`get_rag_response` implementada con patrones asíncronos para consulta y LLM.

Validación y Robustez · 40%

Modelos Pydantic con fuentes, y pruebas ante preguntas fuera de contexto.

Síntesis del módulo

Del texto al vector

Pasamos de ver el texto como caracteres a verlo como un punto en un espacio geométrico.

- **Los embeddings traducen lenguaje a coordenadas; la similitud coseno mide qué tan cerca están dos ideas.**
- **El chunking con overlap y ChromaDB conforman la memoria de largo plazo de cualquier sistema de IA moderno.**

Esta es la base de la arquitectura RAG: el motor que busca información relevante antes de generar una respuesta.

Diagnóstico y optimización

Sesión en Vivo 3

En la sesión en vivo diagnosticamos por qué un RAG devuelve resultados irrelevantes: ¿embedding, chunk size o métrica?

- **Comparamos estrategias de chunking sobre documentos "rebeldes": contratos legales, manuales técnicos y logs de chat.**
- **Optimizamos la persistencia en ChromaDB con metadatos y filtros para aplicaciones escalables.**

Traé una pregunta o "bug" que hayas encontrado en las unidades de código para resolverlo en vivo.



¿Preguntas?

Glosario

- **Embedding:** vector numérico que captura el significado de un texto.
- **One-Hot Encoding:** representación discreta donde cada palabra es una posición con un solo 1.
- **Similitud Coseno:** mide el ángulo entre dos vectores; ignora su longitud.
- **Distancia Euclídea (L2):** distancia en línea recta entre dos puntos vectoriales.
- **Producto Punto:** combina magnitud y ángulo entre vectores.
- **Modelo contextual:** genera un vector distinto para la misma palabra según su contexto.
- **Domain Shift:** caída de precisión al aplicar un modelo general a un dominio muy especializado.

- **Chunking:** dividir un texto largo en fragmentos manejables y coherentes.
- **Overlap:** repetir el final de un fragmento al inicio del siguiente para no perder contexto.
- **Recursive Character Splitting:** estrategia de corte que prueba párrafos, saltos, espacios y caracteres.
- **Token:** unidad mínima de texto que procesa un modelo; se mide con tiktoken.
- **RAG (Retrieval-Augmented Generation):** arquitectura que recupera contexto antes de generar una respuesta.
- **Grounding:** forzar que la respuesta del LLM se base solo en el contexto recuperado.
- **Top-k:** cantidad de fragmentos más relevantes que se recuperan en una búsqueda.

- **Vector Database:** sistema que almacena, indexa y busca vectores de alta dimensionalidad.
- **ChromaDB:** base de datos vectorial de código abierto con modo de persistencia local.
- **Upsert:** insertar un registro o actualizarlo si el ID ya existe.
- **Metadatos:** etiquetas adjuntas a un vector (autor, fecha, fuente) usadas para filtrar.
- **Semantic Search:** búsqueda basada en significado, no en coincidencia exacta de palabras.
- **LCEL:** LangChain Expression Language, para encadenar retriever, prompt y modelo.

¡Gracias!

Hasta la próxima clase